The Mathematical Content of `diamondAt`

The point

The theorem `diamondAt` is an interchange law for hereditary substitution. It says that the following two operations commute:

1. replace variables from one context D by expressions over D';
2. replace variables from another context S by expressions over S'.

There is one important subtlety. The fillers used to replace the S-variables may themselves contain variables from D. Therefore, if we first substitute D-variables, then the S-fillers must also be transformed by that same substitution. The theorem says that this is the only correction needed.

Notation

We write $\mathcal{E}(\Gamma)$ for expressions in context Γ. If α is an arity, then

$$\Gamma \triangleleft \alpha$$

means the context Γ extended by one binder of arity α. This is written $\Gamma :: \alpha$ in Lean.

The operation $\bowtie$ appends telescopes. We use the following names for the contexts appearing in the theorem:

$$P = pre, \qquad D = dom, \qquad D' = cod, \qquad T = \tau, \qquad S = src, \qquad S' = dst.$$

The list υ of already-opened binders gives a telescope

$$U = \mathsf{T}(\upsilon).$$

In Lean, U is $\mathsf{Tele.ofList}\,\upsilon$.

The expression to which the theorem applies lives in the context

$$P \bowtie D \bowtie T \bowtie S \bowtie U.$$

Thus it may mention variables from five regions:

$$P, \qquad D, \qquad T, \qquad S, \qquad U.$$

An argument family can be viewed as a singleton substitution. In Lean, such a family has type `Expr.Args Omega beta`. If A is an argument family for a head of arity β, write

$$A^\sharp : \langle \beta \rangle \rightsquigarrow \Omega$$

for the substitution defined by

$$A^\sharp(\mathsf{here}(j)) = A_j.$$

This is Lean's `Subst.fromArgs beta Omega A`; conversely, `Subst.toArgs` reads a singleton-domain substitution as its argument family. The beta-like step is then not a new primitive operation: it is ordinary substitution action by $A^\sharp$. Thus

$$(A^\sharp)_\Xi(b)$$

means: act on the filler b by the singleton substitution generated from the argument family A, at passive depth Ξ.

The two substitutions

The first substitution is

$$\sigma : D \rightsquigarrow P \bowtie D'.$$

This means that for every variable $z : D \ni \beta$, we have a filler

$$\sigma(z) \in \mathcal{E}(P \bowtie D' \triangleleft \beta).$$

It replaces D-variables while preserving the prefix P.

The second substitution is

$$\kappa : S \rightsquigarrow P \bowtie D \bowtie T \bowtie S'.$$

Thus for every $x : S \ni \beta$,

$$\kappa(x) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \triangleleft \beta).$$

This replaces S-variables by expressions that still live over D. So κ is a substitution before the change of target context induced by σ.

Action at a passive depth

If A is a passive telescope, then σ acts at depth A as a map

$$\sigma_A : \mathcal{E}(P \bowtie D \bowtie A) \longrightarrow \mathcal{E}(P \bowtie D' \bowtie A).$$

The word "passive" means that variables from A are not substituted. They are merely carried along. Only variables from D are active for σ.

Similarly, κ acts at passive depth U as a map

$$\kappa_U : \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U) \longrightarrow \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U).$$

Only variables from S are active for κ; everything else is passive.

Postcomposition of a substitution

Since the fillers of κ may contain D-variables, applying σ to an expression also transforms the substitution κ. We write this postcomposition as

$$\sigma \overset{T \bowtie S'}{\circ} \kappa : S \rightsquigarrow P \bowtie D' \bowtie T \bowtie S'.$$

It is defined pointwise by

$$(\sigma \overset{T \bowtie S'}{\circ} \kappa)(x) = \sigma_{T \bowtie S' \triangleleft \beta}(\kappa(x)), \qquad x : S \ni \beta.$$

The superscript $T \bowtie S'$ records the passive depth at which σ postcomposes κ. In the pointwise definition the full passive depth is $T \bowtie S' \triangleleft \beta$, because the filler $\kappa(x)$ is an expression over

$$P \bowtie D \bowtie T \bowtie S' \triangleleft \beta.$$

After σ acts, this becomes an expression over

$$P \bowtie D' \bowtie T \bowtie S' \triangleleft \beta,$$

which is exactly the required type of a filler for x in the transformed substitution.

The theorem as an equation

Let

$$e \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U).$$

Then `diamondAt` says:

$$\boxed{(\sigma \overset{T \bowtie S'}{\circ} \kappa)_U\big(\sigma_{T \bowtie S \bowtie U}(e)\big) = \sigma_{T \bowtie S' \bowtie U}\big(\kappa_U(e)\big).}$$

This is the clean mathematical statement. The left-hand side first applies σ to e, then substitutes the remaining S-variables using the postcomposition $\sigma \overset{T \bowtie S'}{\circ} \kappa$. The right-hand side first substitutes the S-variables using κ, then applies σ.

The same equation as a square

The theorem says that the following square commutes:

$$\begin{array}{ccc} \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U) & \xrightarrow{\sigma_{T \bowtie S \bowtie U}} & \mathcal{E}(P \bowtie D' \bowtie T \bowtie S \bowtie U) \\ \downarrow \kappa_U & & \downarrow (\sigma \overset{T \bowtie S'}{\circ} \kappa)_U \\ \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U) & \xrightarrow{\sigma_{T \bowtie S' \bowtie U}} & \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U). \end{array}$$

The top arrow changes D into D'. The left arrow changes S into S'. The bottom arrow changes D into D' after S has already been changed into S'. The right arrow changes S into S' after the fillers for S have themselves been postcomposed by σ at passive depth $T \bowtie S'$.

Why the theorem is not trivial

If the fillers of κ did not contain variables from D, the theorem would be close to ordinary commutation of independent substitutions. But here the fillers of κ are expressions over

$$P \bowtie D \bowtie T \bowtie S',$$

so they may contain D-variables. Therefore substituting D-variables before substituting S-variables changes the S-fillers themselves.

The postcomposition $\sigma \overset{T \bowtie S'}{\circ} \kappa$ is precisely the bookkeeping that accounts for this dependency. The theorem says:

the dependency of κ on D is handled pointwise by σ.

The five head cases

The context of e decomposes into five regions:

$$P \bowtie D \bowtie T \bowtie S \bowtie U.$$

Consequently, the head variable of an application in e belongs to exactly one of:

$$U, \qquad S, \qquad T, \qquad D, \qquad P.$$

The proof of the theorem is a structural recursion on e, with a case split on these five possibilities.

Head in U. This is a local binder introduced by recursive descent. Neither substitution acts on it. Both sides rebuild the same head and use the induction hypothesis on all arguments.

More explicitly, suppose the expression has the form

$$e = \mathrm{ap}(x_\upsilon, a),$$

where $x_\upsilon : U \ni \beta$ is the head variable and, for each binder argument $j \in \mathsf{B}(\beta)$, the expression $a(j)$ is the corresponding subterm under the extra binder of arity $\mathsf{ar}(j)$.

After unfolding the two routes in the diamond, both sides still have the same head x_υ. What changes is only the family of arguments. For each argument j, define

$$L_j = (\sigma \overset{T \bowtie S'}{\circ} \kappa)_{U \lhd \mathsf{ar}(j)}\big(\sigma_{T \bowtie S \bowtie U \lhd \mathsf{ar}(j)}(a(j))\big)$$

and

$$R_j = \sigma_{T \bowtie S' \bowtie U \lhd \mathsf{ar}(j)}\big(\kappa_{U \lhd \mathsf{ar}(j)}(a(j))\big).$$

The induction hypothesis applied to the subterm $a(j)$ says precisely that

$$L_j = R_j$$

for every binder argument j. More precisely, for each j we call the same theorem with the same $P, D, D', T, S, S', \sigma, \kappa$, but with the under-telescope enlarged from U to

$$U_j = U \lhd \mathsf{ar}(j),$$

and with expression $a(j)$. In Lean, using ASCII names for readability, this is the recursive call

```
diamondAt (upsilon := j.arity ::  upsilon)
    sigma hTauSrc hTauDst srcTarget dstTarget kappa (args j)
```

Therefore the under case concludes by congruence of the application constructor and function extensionality:

$$\mathrm{ap}(x_\upsilon, j \mapsto L_j) = \mathrm{ap}(x_\upsilon, j \mapsto R_j).$$

So the informal phrase "the under case is $\mathrm{ap}(x_\upsilon, j \mapsto$ induction hypothesis)" means: after both routes have been normalized to applications with the same head, the desired equality is induced pointwise from the recursive equalities on the argument family. In the Lean file, this argument is packaged by the lemma `DiamondSite.underBranch`.

Head in S. This is an active variable for κ. On the right side, κ fires immediately. On the left side, σ first acts on the ambient expression, and then the postcomposition $\sigma \overset{T\bowtie\!\triangleleft S'}{\circ} \kappa$ fires. The two results are equal because $\sigma \overset{T\bowtie\!\triangleleft S'}{\circ} \kappa$ was defined by applying σ to the fillers of κ. This branch recursively invokes the same theorem on the selected filler $\kappa(x)$.

We now spell this out slowly, with the types visible. Suppose

$$e = \mathrm{ap}(x_S, a), \qquad x_S : S \ni \beta.$$

The symbol j below always ranges over the binders of the arity β:

$$j : \mathsf{B}(\beta).$$

The argument family a therefore has components

$$a(j) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U \triangleleft \mathsf{ar}(j)).$$

Write

$$U_j = U \triangleleft \mathsf{ar}(j).$$

Thus $a(j) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U_j)$.

For readability, set

$$\rho = \sigma \overset{T\bowtie\!\triangleleft S'}{\circ} \kappa : S \rightsquigarrow P \bowtie D' \bowtie T \bowtie S'.$$

This is the postcomposition of κ by σ. For a slot $x : S \ni \gamma$, its filler is

$$\rho(x) = \sigma_{T\bowtie\!\triangleleft S'\triangleleft\gamma}(\kappa(x)) \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \triangleleft \gamma).$$

In particular,

$$\kappa(x_S) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \triangleleft \beta),$$

and

$$\rho(x_S) = \sigma_{T\bowtie\!\triangleleft S'\triangleleft\beta}(\kappa(x_S)) \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \triangleleft \beta).$$

With this notation, the branch goal is exactly

$$\rho_U\big(\sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U}(\mathrm{ap}(x_S, a))\big) = \sigma_{T\bowtie\!\triangleleft S'\bowtie\!\triangleleft U}\big(\kappa_U(\mathrm{ap}(x_S, a))\big).$$

Both sides live in

$$\mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

First unfold the left-hand side. The head x_S is passive for σ, because σ only replaces variables from D. Therefore σ preserves the head and acts on the arguments:

$$\sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U}(\mathrm{ap}(x_S, a)) = \mathrm{ap}\Big(x_S, j \mapsto \sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U\triangleleft\mathsf{ar}(j)}(a(j))\Big).$$

Write

$$\bar{a}_j = \sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U_j}(a(j)).$$

This is well-typed because

$$\sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U_j} : \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U_j) \to \mathcal{E}(P \bowtie D' \bowtie T \bowtie S \bowtie U_j),$$

and $a(j)$ is in the domain. Hence

$$\bar{a}_j \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S \bowtie U_j).$$

Then the previous equation says

$$\sigma_{T\bowtie\!\triangleleft S\bowtie\!\triangleleft U}(\mathrm{ap}(x_S, a)) = \mathrm{ap}(x_S, j \mapsto \bar{a}_j).$$

Now apply ρ_U. This substitution acts on x_S, so it fires. Firing means: take the filler $\rho(x_S)$ and substitute into its freshly opened β-binder using the transformed argument family. Define

$$L_j = \rho_{U_j}(\bar{a}_j).$$

This is well-typed because

$$\rho_{U_j} : \mathcal{E}(P \bowtie D' \bowtie T \bowtie S \bowtie U_j) \to \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U_j),$$

and $\bar{a}_j$ is in the domain. Therefore

$$L_j \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U_j).$$

The family $j \mapsto L_j$ gives a singleton-domain substitution

$$L^\sharp : \langle \beta \rangle \rightsquigarrow P \bowtie D' \bowtie T \bowtie S' \bowtie U.$$

Its action at empty passive depth has type

$$(L^\sharp)_\varnothing : \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \triangleleft \beta) \to \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

The input $\rho(x_S)$ has exactly the required domain type $\mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \triangleleft \beta)$. Hence

$$(L^\sharp)_\varnothing(\rho(x_S)) \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

So the left route unfolds as

$$\rho_U\big(\mathrm{ap}(x_S, j \mapsto \bar{a}_j)\big) = (L^\sharp)_\varnothing(\rho(x_S)) = (L^\sharp)_\varnothing\big(\sigma_{T \bowtie S' \triangleleft \beta}(\kappa(x_S))\big).$$

Now unfold the right-hand side. Here κ fires immediately at the head x_S. The filler being fired is $\kappa(x_S)$, and we already recorded that

$$\kappa(x_S) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \triangleleft \beta).$$

Define the pointwise action of κ on the original argument family by

$$(\kappa_U \cdot a)(j) = \kappa_{U_j}(a(j)).$$

This is well-typed because

$$\kappa_{U_j} : \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U_j) \to \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U_j),$$

and $a(j)$ is in the domain. Hence

$$(\kappa_U \cdot a)(j) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U_j).$$

Its sharp is the singleton substitution

$$(\kappa_U \cdot a)^\sharp : \langle \beta \rangle \rightsquigarrow P \bowtie D \bowtie T \bowtie S' \bowtie U$$

defined by

$$(\kappa_U \cdot a)^\sharp(\mathsf{here}(j)) = \kappa_{U_j}(a(j)).$$

The action at empty passive depth has type

$$((\kappa_U \cdot a)^\sharp)_\varnothing : \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \triangleleft \beta) \to \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U).$$

Its input $\kappa(x_S)$ has exactly this domain type, so

$$((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S)) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U).$$

Thus

$$\kappa_U(\mathrm{ap}(x_S, a)) = ((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S)).$$

Applying $\sigma_{T \bowtie S' \bowtie U}$ is now well-typed, since

$$\sigma_{T \bowtie S' \bowtie U} : \mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U) \to \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

So the right route is

$$\sigma_{T \bowtie S' \bowtie U}\left(((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S))\right) \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

In Lean, the singleton substitution $(\kappa_U \cdot a)^\sharp$ is `Subst.actPointwise kappa upsilon args`.

At this point the left route is

$$(L^\sharp)_\varnothing\big(\sigma_{T \bowtie S' \triangleleft \beta}(\kappa(x_S))\big),$$

and the right route is

$$\sigma_{T \bowtie S' \bowtie U}\left(((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S))\right).$$

The recursive call that compares these is `diamondAt` applied to the selected filler $\kappa(x_S)$. Its parameters are

$$T_{\mathrm{new}} = T \bowtie S', \qquad S_{\mathrm{new}} = \langle\beta\rangle, \qquad S'_{\mathrm{new}} = U, \qquad U_{\mathrm{new}} = \varnothing,$$

with expression

$$\kappa(x_S) \in \mathcal{E}(P \bowtie D \bowtie T_{\mathrm{new}} \bowtie S_{\mathrm{new}}).$$

The substitution supplied to this recursive call is

$$(\kappa_U \cdot a)^\sharp : S_{\mathrm{new}} \rightsquigarrow P \bowtie D \bowtie T_{\mathrm{new}} \bowtie S'_{\mathrm{new}},$$

which is the same type as

$$\langle\beta\rangle \rightsquigarrow P \bowtie D \bowtie T \bowtie S' \bowtie U.$$

In Lean, this is the call

```
diamondAt (tau := tau bowtie dst) (src := singleton beta)
    (dst := Tele.ofList upsilon) sigma
    (upsilon := []) ...  (Subst.actPointwise kappa upsilon args)
    (e := kappa xsrc)
```

where `Subst.actPointwise kappa upsilon args` denotes $(\kappa_U \cdot a)^\sharp$.

Let us now unfold what this recursive call says before simplifying its notation. The general diamond theorem, applied with

$$T_{\mathrm{new}} = T \bowtie S', \quad S_{\mathrm{new}} = \langle\beta\rangle, \quad S'_{\mathrm{new}} = U, \quad U_{\mathrm{new}} = \varnothing,$$

says:

$$\left(\sigma \overset{T_{\mathrm{new}} \bowtie S'_{\mathrm{new}}}{\circ} (\kappa_U \cdot a)^\sharp\right)_\varnothing \left(\sigma_{T_{\mathrm{new}} \bowtie S_{\mathrm{new}}}(\kappa(x_S))\right)$$
$$= \sigma_{T_{\mathrm{new}} \bowtie S'_{\mathrm{new}}}\left(((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S))\right).$$

Now substitute the definitions of the new parameters. Since

$$T_{\mathrm{new}} \bowtie S_{\mathrm{new}} = T \bowtie S' \triangleleft \beta, \qquad T_{\mathrm{new}} \bowtie S'_{\mathrm{new}} = T \bowtie S' \bowtie U,$$

the recursive equation becomes

$$\left(\sigma \overset{T\bowtie S'\bowtie U}{\circ} (\kappa_U \cdot a)^\sharp\right)_\varnothing \left(\sigma_{T\bowtie S'\triangleleft\beta}(\kappa(x_S))\right) = \sigma_{T\bowtie S'\bowtie U}\left(((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S))\right).$$

The right-hand side is already the right-hand side we want. The left-hand side still contains the postcomposition

$$\sigma \overset{T\bowtie S'\bowtie U}{\circ} (\kappa_U \cdot a)^\sharp.$$

This is a singleton-domain substitution

$$\langle\beta\rangle \rightsquigarrow P \bowtie D' \bowtie T \bowtie S' \bowtie U.$$

Its filler at $\mathsf{here}(j)$ is, by definition of postcomposition,

$$\begin{aligned}\left(\sigma \overset{T\bowtie S'\bowtie U}{\circ} (\kappa_U \cdot a)^\sharp\right)(\mathsf{here}(j)) &= \sigma_{T\bowtie S'\bowtie U_j}\left((\kappa_U \cdot a)^\sharp(\mathsf{here}(j))\right) \\ &= \sigma_{T\bowtie S'\bowtie U_j}\big(\kappa_{U_j}(a(j))\big).\end{aligned}$$

This is exactly the expression that we call R_j below. Thus the postcomposed singleton substitution is $R^\sharp$. Consequently its action at empty passive depth is

$$(R^\sharp)_\varnothing : \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \triangleleft \beta) \to \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U).$$

The input

$$\sigma_{T\bowtie S'\triangleleft\beta}(\kappa(x_S))$$

has precisely this domain type, because $\kappa(x_S)$ lives in $\mathcal{E}(P \bowtie D \bowtie T \bowtie S' \triangleleft \beta)$. Therefore the left-hand side of the recursive diamond is exactly

$$(R^\sharp)_\varnothing\big(\sigma_{T\bowtie S'\triangleleft\beta}(\kappa(x_S))\big).$$

So, after naming this postcomposed singleton substitution $R^\sharp$, the recursive theorem says exactly that

$$(R^\sharp)_\varnothing\big(\sigma_{T\bowtie S'\triangleleft\beta}(\kappa(x_S))\big) = \sigma_{T\bowtie S'\bowtie U}\left(((\kappa_U \cdot a)^\sharp)_\varnothing(\kappa(x_S))\right),$$

where

$$R_j = \sigma_{T\bowtie S'\bowtie U_j}\big(\kappa_{U_j}(a(j))\big).$$

This is well-typed because $\kappa_{U_j}(a(j))$ lives in $\mathcal{E}(P \bowtie D \bowtie T \bowtie S' \bowtie U_j)$, so

$$R_j \in \mathcal{E}(P \bowtie D' \bowtie T \bowtie S' \bowtie U_j).$$

Similarly, $R^\sharp$ is the singleton substitution generated by $j \mapsto R_j$:

$$R^\sharp : \langle\beta\rangle \rightsquigarrow P \bowtie D' \bowtie T \bowtie S' \bowtie U.$$

The only remaining mismatch is that our unfolded left route contains L_j, not R_j. But the recursive hypotheses already computed for the original arguments $a(j)$ say precisely that. Fix one binder

$$j : \mathsf{B}(\beta).$$

The j-th argument is the smaller expression

$$a(j) \in \mathcal{E}(P \bowtie D \bowtie T \bowtie S \bowtie U_j).$$

The ordinary argument-recursive call of `diamondAt` is made with the same substitutions σ and κ, the same telescopes

$$P, \quad D, \quad D', \quad T, \quad S, \quad S',$$

but with the under-telescope changed from U to

$$U_j = U \triangleleft \mathsf{ar}(j),$$

and with expression $a(j)$. In Lean this family of recursive calls is

```
hargs j := diamondAt (upsilon := j.arity ::  upsilon)
    sigma hTauSrc hTauDst srcTarget dstTarget kappa (args j)
```

Mathematically, this instance of the theorem says

$$\rho_{U_j}\big(\sigma_{T \bowtie S \bowtie U_j}(a(j))\big) = \sigma_{T \bowtie S' \bowtie U_j}\big(\kappa_{U_j}(a(j))\big).$$

The left-hand side is exactly L_j, because

$$\bar{a}_j = \sigma_{T \bowtie S \bowtie U_j}(a(j)) \quad \text{and} \quad L_j = \rho_{U_j}(\bar{a}_j).$$

The right-hand side is exactly R_j, by the definition of R_j. Hence `hargs j` is precisely the equality

$$L_j = R_j$$

for this j.

The equality named `hSubst` in Lean is the extensional equality between the two singleton-domain substitutions generated by the two argument families:

$$L^\sharp = R^\sharp.$$

Both substitutions have domain $\langle \beta \rangle$ and codomain

$$P \bowtie D' \bowtie T \bowtie S' \bowtie U.$$

To prove two such substitutions equal, it is enough to check their value at every singleton slot. There is only one nonempty form of such a slot:

$$\mathsf{here}(j), \qquad j : \mathsf{B}(\beta).$$

At that slot, the desired equality is exactly

$$L^\sharp(\mathsf{here}(j)) = R^\sharp(\mathsf{here}(j)),$$

which unfolds to $L_j = R_j$, i.e. exactly `hargs j`. This is why the Lean proof of `hSubst` has one real case, $\mathsf{here}(j)$, and that case is closed by `hargs j`.

This recursive call is terminating because it decreases the source side of the diamond: instead of proving the theorem for all of S, it proves it for the single selected S-slot x_S.

Head in T. This head is passive for both substitutions. Both sides preserve it and recurse on the arguments.

Concretely, suppose

$$e = \mathrm{ap}(x_T, a),$$

where $x_T : T \ni \beta$. This head lies in the ambient passive telescope between the D-part and the S-part. It is not a variable that σ may replace, because σ only acts on D. It is also not a variable that κ may replace, because κ only acts on S. Thus both routes through the diamond rebuild the same T-head.

For each binder argument $j \in \mathsf{B}(\beta)$, set

$$L_j = (\sigma \overset{T \bowtie\!\lhd S'}{\circ} \kappa)_{U \lhd \mathsf{ar}(j)}\big(\sigma_{T \bowtie\!\lhd S \bowtie\!\lhd U \lhd \mathsf{ar}(j)}(a(j))\big)$$

and

$$R_j = \sigma_{T \bowtie\!\lhd S' \bowtie\!\lhd U \lhd \mathsf{ar}(j)}\big(\kappa_{U \lhd \mathsf{ar}(j)}(a(j))\big).$$

The recursive hypothesis again gives $L_j = R_j$ for every j. The recursive call is exactly the same one as in the U-case: for each argument j, we apply the theorem to the subterm $a(j)$, leaving σ, κ, T, S, S' and all coherence data unchanged, and replacing the under-telescope U by

$$U_j = U \lhd \mathsf{ar}(j).$$

In Lean, again with ASCII names for readability, this is

```
diamondAt (upsilon := j.arity ::  upsilon)
    sigma hTauSrc hTauDst srcTarget dstTarget kappa (args j)
```

Therefore the T-case reduces to

$$\mathrm{ap}(x_T, j \mapsto L_j) = \mathrm{ap}(x_T, j \mapsto R_j).$$

The Lean proof is longer than this sentence because it must show that the formal embeddings of the same T-head through $P \bowtie D \bowtie T \bowtie S$ and $P \bowtie D \bowtie T \bowtie S'$ are the intended ones. Once this reassociation bookkeeping is done, the branch is just congruence plus the recursive equalities.

Head in D. This is an active variable for σ. Firing σ creates a one-binder instantiation from the arguments of the application. This is the only branch where the companion theorem for one-binder instantiation is needed.

Head in P. This is part of the untouched prefix. Both substitutions preserve it and recurse on the arguments.

Here an application has the form

$$e = \mathrm{ap}(x_P, a),$$

with $x_P : P \ni \beta$. The prefix P is shared background context: σ preserves it while replacing only D-variables, and κ preserves it while replacing only S-variables. Consequently the head x_P survives unchanged along both sides of the square.

As before, for each binder argument j we compare the two transformed subterms

$$L_j = (\sigma \overset{T \bowtie\!\lhd S'}{\circ} \kappa)_{U \lhd \mathsf{ar}(j)}\big(\sigma_{T \bowtie\!\lhd S \bowtie\!\lhd U \lhd \mathsf{ar}(j)}(a(j))\big), \qquad R_j = \sigma_{T \bowtie\!\lhd S' \bowtie\!\lhd U \lhd \mathsf{ar}(j)}\big(\kappa_{U \lhd \mathsf{ar}(j)}(a(j))\big).$$

The induction hypothesis gives $L_j = R_j$. Concretely, for each j the recursive call is the theorem applied to $a(j)$ at the enlarged passive under-telescope

$$U_j = U \lhd \mathsf{ar}(j),$$

with the same substitutions σ and κ, the same ambient telescope T, the same source and destination S, S', and the same coherence witnesses. In Lean, using ASCII names for readability, it is the same call

```
diamondAt (upsilon := j.arity ::  upsilon)
    sigma hTauSrc hTauDst srcTarget dstTarget kappa (args j)
```

Hence

$$\mathrm{ap}(x_P, j \mapsto L_j) = \mathrm{ap}(x_P, j \mapsto R_j).$$

The only formal work in this case is showing that the prefix injection for x_P is the same after passing through the source and destination parenthesizations. Mathematically, nothing happens to the head.

Why the formal statement has coherence data

The mathematical equation above suppresses associativity bookkeeping for $\bowtie$. Lean cannot suppress it completely. The formal theorem therefore carries witnesses saying that the decompositions

$$T \bowtie S \qquad \text{and} \qquad T \bowtie S'$$

are proper and that injections into larger contexts compute with the intended parenthesization. These witnesses do not change the mathematical content of the theorem. They only ensure that the formal proof can recognize, for example, that a head from S remains a head from S after it is embedded through

$$P \bowtie D \bowtie T \bowtie S.$$

Role in substitution composition

The public composition theorem says that acting by a composite substitution is the same as acting by one substitution and then the other. The difficult case is when the first substitution fires on an application head. Firing creates a fresh one-binder instantiation, and the remaining substitution must commute past that instantiation.

The theorem described here is the general interchange principle that makes that step possible. In short:

hereditary substitution respects substitution composition.